Atividade Avaliativa – Implementação de Algoritmos de Ordenação em C

1. Introdução

Este relatório apresenta a implementação e análise comparativa de cinco algoritmos de ordenação desenvolvidos na linguagem C, que foram trabalhados, explicados e usados em sala de aula:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Merge Sort

2. Ambiente e Estrutura do Programa

O programa foi desenvolvido em linguagem C utilizando as seguintes bibliotecas: stdio.h, stdlib.h, time.h, locale.h. Foi definido o tamanho do array logo no início para padronizar os arrays usados e facilitar para a modificação do tamanho caso seja necessário e oportuno.

Vetores Utilizados nos Testes

Foram utilizados três tipos de vetores para avaliar o comportamento dos algoritmos:

1. **Vetor aleatório**
   {47, 82, 15, 63, 94, 28, 71, 39, 56, 20}
2. **Vetor já ordenado (crescente)**
   {15, 20, 28, 39, 47, 56, 63, 71, 82, 94}
3. **Vetor ordenado decrescente (pior caso para alguns algoritmos)**
   {94, 82, 71, 63, 56, 47, 39, 28, 20, 15}

3. Implementação dos Algoritmos

Bubble Sort

O Bubble Sort realiza comparações entre elementos adjacentes e troca suas posições caso estejam fora de ordem.

A cada comparação, o maior elemento é deslocado para o final do vetor.

Complexidade:

- Melhor caso: $O(n^2)$

- Médio caso: $O(n^2)$
- Pior caso: $O(n^2)$

Selection Sort

O Selection Sort percorre o vetor procurando o menor elemento da parte não ordenada e o troca com a posição atual.

Complexidade:

- Melhor caso: $O(n^2)$
- Médio caso: $O(n^2)$
- Pior caso: $O(n^2)$

Insertion Sort

O Insertion Sort percorre o vetor da esquerda para a direita, inserindo cada elemento na posição correta entre os já ordenados.

Complexidade:

- Melhor caso: $O(n)$
- Médio caso: $O(n^2)$
- Pior caso: $O(n^2)$

Quick Sort

O Quick Sort é um algoritmo recursivo que:

1. Escolhe um pivô (no código, o último elemento do vetor).
2. Particiona o vetor em:
    - Elementos menores ou iguais ao pivô
    - Elementos maiores que o pivô
3. Aplica recursivamente o mesmo processo nas duas partições.

Funções auxiliares utilizadas:

- troca()
- partir()

Complexidade:

- Melhor caso: $O(n \log n)$

- Médio caso: O(n log n)
- Pior caso: $O(n^2)$

Merge Sort

O Merge Sort utiliza a técnica de divisão e conquista.

Funcionamento:

1. Divide o vetor ao meio recursivamente.
2. Ordena as duas metades.
3. Realiza a intercalação ordenada (merge).

Foi utilizada alocação dinâmica de memória com malloc() para criar vetores auxiliares temporários.

Complexidade:

- Melhor caso: O(n log n)
- Médio caso: O(n log n)
- Pior caso: O(n log n)

4. Medição de Desempenho

A medição foi feita utilizando o tempo de execução medido pelo função clock().

O tempo foi convertido para segundos com:

((double)t) / CLOCKS_PER_SEC

Como o vetor é definido com apenas 10 elementos o tempo de execução é mínimo aparecendo quase ou totalmente zerados.

5. Resultados Observados

Com base nos testes realizados:

- Todos os algoritmos conseguiram ordenar os três vetores.
- Para vetores pequenos como o utilizado no código a diferença de tempo é quase imperceptível.
- Insertion Sort apresentou melhor desempenho no vetor já ordenado.
- Quick Sort pode apresentar pior desempenho no vetor já ordenado devido à escolha do pivô.
- Merge Sort manteve desempenho estável em todos os casos.

6. Comparação de Desempenho

Algoritmo	Melhor Caso	Pior Caso	Estabilidade	Uso de Memória
Bubble Sort	$O(n^2)$	$O(n^2)$	Estável	Baixo
Selection Sort	$O(n^2)$	$O(n^2)$	Não estável	Baixo
Insertion Sort	$O(n)$	$O(n^2)$	Estável	Baixo
Quick Sort	$O(n \log n)$	$O(n^2)$	Não estável	Médio
Merge Sort	$O(n \log n)$	$O(n \log n)$	Estável	Alto

Principais Conclusões da Comparação

- Para vetores pequenos, todos funcionam adequadamente.
- Para vetores grandes:
  - Merge Sort e Quick Sort são mais eficientes, justamente pela divisão dos vetores.
  - Bubble e Selection são menos recomendados, já que fazem muitas comparações para conseguirem se organizarem.
- Insertion Sort é eficiente para vetores quase ordenados.

7. Conclusão

A atividade permitiu compreender a implementação prática de diferentes algoritmos de ordenação e comparar seus comportamentos em diferentes cenários.

Embora todos os algoritmos tenham funcionado corretamente, algoritmos baseados em divisão e conquista Quick Sort e Merge Sort apresentam melhor desempenho teórico para grandes volumes de dados.

Já algoritmos simples como Bubble Sort e Selection Sort são mais indicados para fins didáticos.

Podemos também citar alguns exemplo em que esses algoritmos são utilizados na vida real:

- Bubble Sort – Usado para organizar pequenas listas em sistemas simples ou embarcados..
- Selection Sort – Usado em situações onde o número de trocas precisa ser mínimo, como em sistemas onde escrever na memória é custoso.

- Insertion Sort – Usado para ordenar pequenas quantidades de dados ou listas que já estão quase ordenadas, como organizar cartas na mão ou atualizar uma lista que recebe novos elementos aos poucos.

- Quick Sort – Usado em bibliotecas padrão de programação e sistemas que precisam ordenar grandes volumes de dados rapidamente..

- Merge Sort – Usado em ordenação de grandes volumes de dados que não cabem totalmente na memória, como processamento de arquivos muito grandes e sistemas de Big Data.